

# Replicação Master-Replica com Redis em Ambiente Virtualizado

**Disciplina:** Sistemas Distribuídos

**Ambiente:** Ubuntu Server 26.04 LTS no Oracle VirtualBox

**Autor:** Thomaz Arthur e Lucas Gabriel

## 1. Visão Geral da Arquitetura

O objetivo desta prática foi implementar e validar uma arquitetura distribuída de banco de dados em memória utilizando o Redis. Em cenários reais de microsserviços e sistemas de alto tráfego, manter uma única instância de banco centralizada gera um Ponto Único de Falha (*Single Point of Failure* - *SPOF*) e limita a vazão das consultas à capacidade de hardware de uma única máquina.

Para resolver esse gargalo, estruturamos um cluster com topologia primário-secundário (Master-Replica) composto por dois nós virtualizados:

- **REDIS01 (Master):** Responsável por processar todas as operações de escrita (**SET**) e replicar os dados para a réplica.
- **REDIS02 (Replica):** Configurado em modo de leitura estrita (*read-only*), recebendo as atualizações do Master em tempo real e atendendo consultas (**GET**).

A replicação assíncrona reduz a carga de I/O sobre o nó principal, distribui o volume de leituras e garante a integridade dos dados caso seja necessário promover a réplica em uma falha de hardware ou rede.

## 2. Fundamentos Teóricos

### 2.1. Replicação Assíncrona e Mecanismo PSYNC

O Redis realiza a replicação de dados em segundo plano sem bloquear o processamento de novas requisições no Master. Assim que uma escrita é confirmada localmente, o comando entra em um buffer circular (*replication backlog*) e é enviado como fluxo contínuo para as réplicas conectadas.

- **Sincronização Inicial (Full Sync):** Quando a réplica conecta pela primeira vez, o Master gera um snapshot da memória em disco (formato RDB) e o transmite via socket para que a réplica monte o estado base.
- **Sincronização Parcial (PSYNC):** Em caso de oscilação momentânea de rede, a réplica reconecta informando o seu deslocamento de dados (*offset*). O Master reenvia apenas os bytes pendentes acumulados no backlog, sem necessidade de transferir um novo arquivo RDB completo.

### 2.2. Consistência e Teorema CAP

Pelo Teorema CAP, a arquitetura Redis em modo de replicação padrão prioriza **Disponibilidade (A)** e **Tolerância a Partições de Rede (P)**, operando sob o modelo de **Consistência Eventual**:

- A confirmação da escrita é devolvida ao cliente antes da confirmação da réplica, mantendo a latência baixa.
- Existe uma janela mínima de propagação (*replication lag*). Caso o Master sofra uma parada forçada antes de esvaziar o buffer de replicação, dados recentes não propagados podem ser perdidos durante um failover, refletindo a troca consciente entre desempenho extremo e consistência estrita.

### 2.3. Tolerância a Falhas e Recuperação (Failover)

A redundância física de dados em duas instâncias isoladas permite a continuidade do serviço. Caso o nó Master fique inacessível, a réplica é desvinculada e promovida a Master (*replicaof no one*), assumindo a recepção de escritas e restabelecendo o sistema.

## 3. Topologia e Configurações de Rede

Para isolar o tráfego do cluster do restante da rede do hospedeiro, cada máquina virtual recebeu duas placas de rede no VirtualBox:

1. **Adaptador 1 (NAT)**: Concede acesso à internet para download de pacotes via *apt*.
2. **Adaptador 2 (Rede Interna *redis-net*)**: Comunicação exclusiva ponto a ponto entre os servidores do cluster.

| Máquina | Função  | IP da Rede Interna | Interface | Porta |
|---------|---------|--------------------|-----------|-------|
| REDIS01 | Master  | 192.168.50.10/24   | enp0s8    | 6379  |
| REDIS02 | Replica | 192.168.50.20/24   | enp0s8    | 6379  |

## 4. Roteiro Prático de Configuração

### 4.1. Definição de IPs Estáticos no Netplan

No arquivo */etc/netplan/00-installer-config.yaml* de cada máquina:

- **REDIS01:**
- YAML

```
network:
  version: 2
  ethernet:
    enp0s3:
      dhcp4: true
    enp0s8:
      addresses: [192.168.50.10/24]
```

- **REDIS02:**
- YAML

```
network:
  version: 2
  ethernet:
    enp0s3:
      dhcp4: true
    enp0s8:
      addresses: [192.168.50.20/24]
```

- 

Aplicação dos parâmetros: `sudo netplan apply`.

## 4.2. Parametrização do Redis Master (**REDIS01**)

No arquivo `/etc/redis/redis.conf`:

```
Plaintext
bind 127.0.0.1 -::1 192.168.50.10
protected-mode no
```

Reinicialização do serviço: `sudo systemctl restart redis-server`.

## 4.3. Clonagem e Configuração da Réplica (**REDIS02**)

A réplica foi gerada por clonagem completa no VirtualBox (com geração de novos endereços MAC). Após a inicialização, foram executados os seguintes ajustes:

1. Atualização definitiva do hostname:
2. Bash

```
sudo hostnamectl set-hostname REDIS02
sudo sed -i 's/REDIS01/REDIS02/g' /etc/hosts
sudo sed -i 's/preserve_hostname: false/preserve_hostname: true/g' /etc/cloud/cloud.cfg
```

- 3.
4. No `/etc/redis/redis.conf`:
5. Plaintext

```
bind 127.0.0.1 -:::1 192.168.50.20
protected-mode no
```

- 6.
7. Ativação da replicação:
8. Bash

```
redis-cli replicaof 192.168.50.10 6379
redis-cli config rewrite
9.
```

## 5. Validação dos Cenários Práticos

### Cenário 1: Escrita no Master, Leitura na Réplica e Proteção de Escrita

- **Gravação no Master:**
- Bash

```
redis-cli -h 192.168.50.10 set turma "Sistemas Distribuidos"
# Retorno: OK
```

- 
- **Consulta na Réplica:**
- Bash

```
redis-cli -h 192.168.50.20 get turma
# Retorno: "Sistemas Distribuidos"
```

- 
- **Tentativa de Escrita Indevida na Réplica:**
- Bash

```
redis-cli -h 192.168.50.20 set teste 123
# Retorno: (error) READONLY You can't write against a read only replica.
```

- 
- **Conclusão:** O fluxo de replicação unidirecional funcionou perfeitamente e o modo *read-only* protegeu a base contra inconsistências locais.

### Cenário 2: Simulação de Falha do Nó Principal

- O serviço do Master foi derrubado intencionalmente:
- Bash

```
sudo systemctl stop redis-server
```

- 
- Consulta de diagnóstico na Réplica:
- Bash

```
redis-cli info replication
```

- 
- O campo `master_link_status` mudou imediatamente para **down**, registrando a interrupção da conexão com o primário sem travar os dados já armazenados na memória da réplica.

### Cenário 3: Promoção Manual da Réplica (Failover)

Para restabelecer a escrita no cluster com o Master indisponível:

```
Bash
redis-cli replicaof no one
```

Ao executar `redis-cli info replication`, o nó passou a exibir `role:master`. Um novo comando de gravação foi executado diretamente nele:

```
Bash
redis-cli set status "Recuperado com sucesso"
redis-cli get status
```

Retorno: `"Recuperado com sucesso"`. O cluster manteve a continuidade das operações.

## 6. Script de Automação em Python

Arquivo `teste_cluster.py` para validação programática dos cenários via biblioteca `python3-redis`:

```
Python
#!/usr/bin/env python3
import time
import redis

MASTER_IP = "192.168.50.10"
REPLICA_IP = "192.168.50.20"
PORT = 6379

def main():
    print("=" * 55)
    print("VALIDAÇÃO PRÁTICA: CLUSTER REDIS MASTER-REPLICA")
    print("=" * 55)

    try:
        master = redis.Redis(host=MASTER_IP, port=PORT, decode_responses=True,
socket_timeout=3)
        replica = redis.Redis(host=REPLICA_IP, port=PORT, decode_responses=True,
socket_timeout=3)

        # 1. Escrita no Master
        master.set("disciplina", "Sistemas Distribuidos")
        print(f"[Master] Chave gravada: {master.get('disciplina')}")

        time.sleep(0.5)

        # 2. Leitura na Réplica
        valor = replica.get("disciplina")
```

```

print(f"[Réplica] Leitura sincronizada: {valor}")

# 3. Teste de Violação Read-Only
try:
    replica.set("tentativa", "erro")
except redis.exceptions.ReadOnlyError:
    print(f"[Réplica] Escrita bloqueada com sucesso (Modo Read-Only ativo)")

except redis.exceptions.ConnectionError as err:
    print(f"Falha de conexão com os nós: {err}")

print("=" * 55)

if __name__ == "__main__":
    main()

```

## 7. Registro de Problemas e Soluções (Troubleshooting)

Durante a montagem do laboratório, foram documentadas e tratadas as seguintes ocorrências:

1. **Formatação YAML no Netplan:**
  - *Problema:* Erros de digitação (`etwork:`) e recuo desalinhado na linha `addresses:` impediram o Netplan de aplicar os IPs.
  - *Solução:* Ajuste da indentação hierárquica em dois espaços e simplificação da linha para a sintaxe `addresses: [192.168.50.10/24]`.
2. **Identificador Inválido de Serviço no Systemd:**
  - *Problema:* Tentativa de reiniciar o serviço com `systemctl restart REDIS01`, retornando `Unit REDIS01.service not found`.
  - *Solução:* Correção para o identificador real do pacote: `sudo systemctl restart redis-server`.
3. **Inconsistência de Hostname após Clonagem:**
  - *Problema:* A VM clonada manteve o nome `REDIS01` e o utilitário `cloud-init` reverteria qualquer alteração após reiniciar.
  - *Solução:* Aplicação do `hostnamectl set-hostname REDIS02` e desativação do reset configurando `preserve_hostname: true` em `/etc/cloud/cloud.cfg`.
4. **Isolamento de Rede entre as VMs:**
  - *Problema:* O comando `ping` retornou `Destination Host Unreachable` e o Redis acusou `master_link_status:down`.
  - *Solução:* Identificou-se que o nó Master estava desligado e que a interface `enp0s8` ainda não havia subido o IP no boot. A inicialização simultânea de ambas e a execução do `netplan apply` restabeleceram a comunicação com 0% de perda.
5. **Restrição de Instalação Global do PIP (PEP 668):**
  - *Problema:* O comando `pip install redis` foi abortado pelo Ubuntu com o erro `externally-managed-environment`.
  - *Solução:* Utilização do pacote nativo gerenciado pelo sistema operacional: `sudo apt install -y python3-redis`.